

ESTRUCTURAS DE DATOS

Práctica de Laboratorio N° 09: Implementación de Árboles Binarios de Búsqueda

I. OBJETIVOS DE LA PRÁCTICA:

- Comprender el Árbol Binario de Búsqueda (ABB) como una estructura de datos dinámica, no lineal y recursiva, que permite organizar información jerárquicamente para facilitar operaciones eficientes de búsqueda, inserción y eliminación.
- Comparar las ventajas del ABB frente a otras estructuras de datos, reconociendo su eficiencia y aplicabilidad en la resolución de problemas que requieren acceso rápido y ordenado a los datos.
- Implementar y analizar los recorridos inorden, preorden y posorden, así como algoritmos de inserción y búsqueda en un ABB, utilizando tanto métodos recursivos como iterativos.

II. DESARROLLO DE LA PRÁCTICA:

1. Formalmente se define un árbol binario de búsqueda de la siguiente manera (Cairó Guardati, 2006): Para todo nodo T del árbol se debe cumplir que todos los valores almacenados en el subárbol izquierdo de T sean menores o iguales a la información guardada en el nodo T. De forma similar, todos los valores almacenados en el subárbol derecho de T deben ser mayores o iguales a la información guardada en el nodo T. Los valores a los que se hace referencia en la definición refieren al contenido del campo información del nodo.
2. En esta práctica se implementará un árbol binario de búsqueda (ABB) utilizando nodos creados dinámicamente con punteros. Se creará una clase llamada Arbol que almacenará la raíz del árbol, y se desarrollarán los siguientes métodos:
 - **insertarNodo()** – Inserta un nuevo nodo en el árbol usando un enfoque recursivo, respetando las propiedades del ABB.
 - **insertarNodo()** – Inserta un nuevo nodo en el árbol usando un enfoque recursivo, respetando las propiedades del ABB. Este algoritmo considera el caso de un árbol vacío.
 - **insertarNodoIterativo()** – Inserta un nuevo nodo de forma iterativa, comparando y ubicando el nodo en su lugar correspondiente.
 - **buscarABB()** – Busca un valor dentro del árbol utilizando recursividad. Retorna si el valor existe o no.
 - **buscarABB2()** – Busca un valor dentro del árbol utilizando recursividad. Retorna si el valor existe o no. Versión 2.
 - **buscarABBIterativo()** – Realiza una búsqueda iterativa para verificar si el valor se encuentra en el ABB.
 - **muestraArbol()** – Escribe en pantalla el árbol en forma horizontal, con la raíz a la izquierda..
 - **preorden()** – Recorre el árbol en preorden (raíz – izquierda – derecha) y muestra los valores de los nodos en ese orden.
 - **inorden()** – Recorre el árbol en inorden (izquierda – raíz – derecha), útil para obtener los elementos ordenados de menor a mayor.
 - **posorden()** – Recorre el árbol en posorden (izquierda – derecha – raíz) y muestra los valores de los nodos en ese orden.
 - **altura()** – Calcula la altura del árbol, es decir, la longitud del camino más largo desde la raíz hasta un nodo hoja.
 - **contarNodos()** – Cuenta el total de nodos que conforman el árbol.

ESTRUCTURAS DE DATOS

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  class Nodo
6  {
7  public:
8      int info;
9      Nodo* izq;
10     Nodo* der;
11 };
12
13 class Arbol
14 {
15 private:
16     Nodo* raiz; //apuntador al inicio del arbol
17
18 public:
19     Arbol(); //constructor
20
21     Nodo* regresaRaiz();
22     void insertarNodo(Nodo*& apnodo, int dato);
23     void insertarNodo2(Nodo*& apnodo, int dato);
24     void insertarNodoIterativo(Nodo*& apnodo, int dato);
25     bool busquedaABB(Nodo* apnodo, int dato);
26     bool busquedaABB2(Nodo* apnodo, int dato);
27     bool busquedaABBIterativa(Nodo* apnodo, int dato);
28     void muestraArbol(Nodo* apnodo, int nivel);
29
30     // Métodos de recorrido
31     void preorden(Nodo* apnodo);
32     void inorden(Nodo* apnodo);
33     void posorden(Nodo* apnodo);
34
35     int altura(Nodo* apnodo);
36     int contarNodos(Nodo* apnodo);
37     int contarHojas(Nodo* apnodo);
38 };
39
40 //constructor predeterminado
41 Arbol::Arbol()
42 {
43     raiz = NULL;
44 }
45
46 //regresa el apuntador a la raiz
47 Nodo *Arbol::regresaRaiz(){
48     return raiz;
49 }
50
```

```
51 void Arbol::insertarNodo(Nodo*& apnodo, int dato)
52 {
53     if(dato < apnodo->info){
54         if(apnodo->izq == NULL){
55             Nodo* otro = new Nodo();
56             otro->info = dato;
57             otro->izq = NULL;
58             otro->der = NULL;
59             apnodo->izq = otro;
60         }else{
61             insertarNodo(apnodo->izq, dato);
62         }
63     }else{
64         if(dato > apnodo->info){
65             if(apnodo->der == NULL){
66                 Nodo* otro = new Nodo();
67                 otro->info = dato;
68                 otro->izq = NULL;
69                 otro->der = NULL;
70                 apnodo->der = otro;
71             }else{
72                 insertarNodo(apnodo->der, dato);
73             }
74         }else{
75             cout << "El nodo " << dato << " ya se encuentra en el árbol." << endl;
76         }
77     }
78 }

80 void Arbol::insertarNodo2(Nodo*& apnodo, int dato)
81 {
100
101 void Arbol::insertarNodoIterativo(Nodo*& apnodo, int dato)
102 {
129
130 bool Arbol::busquedaABB(Nodo* apnodo, int dato)
131 {
153
154 bool Arbol::busquedaABB2(Nodo* apnodo, int dato)
155 {
156     if(apnodo != NULL){
157         if(dato < apnodo->info){
158             busquedaABB2(apnodo->izq, dato);
159         }else{
160             if(dato > apnodo->info){
161                 busquedaABB2(apnodo->der, dato);
162             }else{
163                 //cout << "La información se encuentra en el árbol";
164                 return true;
165             }
166         }
167     }else{
168         //cout << "La información no se encuentra en el árbol";
169         return false;
170     }
171 }

173 bool Arbol::busquedaABBIterativa(Nodo* apnodo, int dato)
174 {
191
192 void Arbol::muestraArbol(Nodo* apnodo, int nivel)
193 {
200
```

ESTRUCTURAS DE DATOS

```
270 void menu()
271 {
272     Arbol arbol;
273     Nodo* raiz = arbol.regresaRaiz();
274     int opcion, dato;
275
276     do {
277         cout << "\n==== MENÚ DE OPCIONES - ÁRBOL BINARIO =====> endl;
278         cout << "1. Insertar nodo (insertarNodo)" << endl;
279         cout << "2. Insertar nodo (insertarNodo2)" << endl;
280         cout << "3. Insertar nodo (insertarNodoIterativo)" << endl;
281         cout << "4. Mostrar árbol (forma estructurada)" << endl;
282         cout << "5. Recorrido en Preorden" << endl;
283         cout << "6. Recorrido en Inorden" << endl;
284         cout << "7. Recorrido en Posorden" << endl;
285         cout << "8. Buscar dato (busquedaABB)" << endl;
286         cout << "9. Altura del árbol" << endl;
287         cout << "10. Contar todos los nodos" << endl;
288         cout << "11. Contar nodos hoja" << endl;
289         cout << "0. Salir" << endl;
290         cout << "Seleccione una opción: ";
291         cin >> opcion;
292
293         switch(opcion) {
294             case 1:
295                 cout << "Ingrese dato a insertar (insertarNodo): ";
296                 cin >> dato;
297                 if (raiz == NULL) {
298                     raiz = new Nodo{dato, NULL, NULL};
299                 } else {
300                     arbol.insertarNodo(raiz, dato);
301                 }
302                 break;
303             case 2:
304                 cout << "Ingrese dato a insertar (insertarNodo2): ";
305                 cin >> dato;
306                 arbol.insertarNodo2(raiz, dato);
307                 break;
308             case 3:
309                 cout << "Ingrese dato a insertar (insertarNodoIterativo): ";
310                 cin >> dato;
311                 arbol.insertarNodoIterativo(raiz, dato);
312                 break;
313             case 4:
314                 if (raiz == NULL)
315                     cout << "El árbol está vacío.\n";
316                 else
317                     arbol.muestraArbol(raiz, 0);
318                 break;
319             case 5:
320                 if (raiz == NULL)
321                     cout << "El árbol está vacío.\n";
322                 else {
323                     cout << "Recorrido Preorden : ";
324                     arbol.preorden(raiz);
325                     cout << endl;
326                 }
327                 break;
328             case 6:
329                 if (raiz == NULL)
330                     cout << "El árbol está vacío.\n";
331                 else {
332                     cout << "Recorrido Inorden : ";
333                     arbol.inorden(raiz);
334                     cout << endl;
335                 }
336                 break;
```

```
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380

case 7:
    if (raiz == NULL)
        cout << "El árbol está vacío.\n";
    else {
        cout << "Recorrido Posorden : ";
        arbol.posorden(raiz);
        cout << endl;
    }
    break;
case 8:
    cout << "Ingrese el dato a buscar: ";
    cin >> dato;
    if (arbol.busquedaABB(raiz, dato))
        cout << "Dato encontrado en el árbol.\n";
    else
        cout << "Dato NO encontrado.\n";
    break;
case 9:
    if (raiz == NULL)
        cout << "El árbol está vacío.\n";
    else
        cout << "Altura del árbol: " << arbol.altura(raiz) << endl;
    break;
case 10:
    if (raiz == NULL)
        cout << "El árbol está vacío.\n";
    else
        cout << "Cantidad total de nodos: " << arbol.contarNodos(raiz) << endl;
    break;
case 11:
    if (raiz == NULL)
        cout << "El árbol está vacío.\n";
    else
        cout << "Cantidad de nodos hoja: " << arbol.contarHojas(raiz) << endl;
    break;
case 0:
    cout << "Saliendo del programa...\n";
    break;
default:
    cout << "Opción inválida. Intente nuevamente.\n";
}

} while(opcion != 0);
}
```

3. Considerando que un árbol binario de búsqueda almacena números enteros, encuentre e imprima el máximo valor guardado en el árbol.
4. Considerando que un árbol binario de búsqueda almacena números enteros, encuentre e imprima el mínimo valor guardado en el árbol.
5. Dadas las siguientes claves que representan los signos del zodiaco, construya un árbol binario de búsqueda.

pisicis - acuario - capricornio - cáncer - sagitario - virgo - leo - escorpión - libra - géminis - aries - tauro